

Documento B – Processo de Teste

Projeto: Contador de Corte para a PackBag (PI - Grupo 8 – Semestre 3)

Tecnologia: Flutter/Dart + Firebase

Arquitetura: MVVM

Norma base: ISO/IEC/IEEE 29119-1

1. Planejamento de Teste

1.1 Estratégia

Serão utilizados:

- Testes de unidade para as classes LoginViewModel, RegisterViewmodel e ContagemCortesService.
- Testes de integração para a classe AuthServices, em interação com o serviço real do Firebase Authentication (projeto de desenvolvimento).

Não serão utilizados:

- Testes de interface do usuário (widgets, telas).
- Testes com backend de produção ou banco de dados real (exceto o Firebase para autenticação, usado em ambiente controlado).
- Testes de desempenho, segurança ou acessibilidade.

A escolha da estratégia visa validar a lógica de negócio isolada (ViewModels, Service) e a comunicação com o componente externo de autenticação, conforme os riscos e o escopo definidos no Documento 01.

2. Ambiente

- SDK: Flutter (canal estável) com flutter test e integration_test.
- Serviço de autenticação: Firebase Authentication, utilizando DefaultFirebaseOptions.currentPlatform.
- Dependências externas: Nenhuma outra dependência real; os ViewModels e o ContagemCortesService são testados isoladamente, sem mocks ou fakes.
- Execução: Linha de comando ou IDE compatível (VS Code / Android Studio), com dispositivo virtual ou navegador para testes de integração.]

3. Critérios de Entrada

- ViewModels (LoginViewmodel, RegisterViewmodel) implementados.
- ContagemCortesService implementado.
- AuthServices implementado com suporte a createAccount, signIn, signOut e resetPassword.
- Requisitos funcionais e condições de teste definidos no Documento 01.
- Configuração do Firebase devidamente inicializada para o ambiente de teste.

4. Critérios de Saída

- Todos os casos de teste executados.
- Resultados registrados (aprovado/reprovado).
- Falhas identificadas e documentadas.
- Relatório final de execução gerado (evidências).

5. Análise de Teste

A análise foi realizada a partir dos requisitos, riscos e escopo definidos no Documento 01, resultando nas Condições de Teste CT01 a CT35. Essas condições representam os comportamentos ou situações que necessitam ser verificados e são a base para a elaboração dos casos de teste.

6. Controle de Teste

6.1 Controle de Execução (exemplo de ciclo)

Indicador	Valor
Total de casos	35
Executados	35
Aprovados	35
Reprovados	0
Não executados	0

6.2 Casos reprovados e análise preliminar:

- TC-33 (AuthServices – Login com senha inválida): Falha intermitente devido a timeout na resposta do Firebase; possível necessidade de ajuste no tempo de espera.
- TC-35 (AuthServices – Reset de senha): Exceção não capturada adequadamente; o teste espera ausência de exceção, mas o método

retornou um erro inesperado. Será necessário rever o tratamento de erro no serviço.

Nota: Os dois casos acima são apenas exemplos criados para ilustrar a documentação em caso de reprovação e não representam os testes reais.

7. Análise de Teste

A análise foi realizada a partir dos requisitos, riscos e escopo definidos no Documento 01, resultando nas Condições de Teste CT01 a CT35. Essas condições representam os comportamentos ou situações que necessitam ser verificados e são a base para a elaboração dos casos de teste.

8. Projeto de Teste

As condições de teste são transformadas em casos de teste, cada um com identificador único, pré-condições, dados de entrada, ações e resultados esperados. A seguir são apresentados exemplos detalhados representativos dos diferentes módulos, e uma tabela de rastreabilidade geral.

8.1 Exemplos de Casos de Teste Detalhados

TC-01 – Cadastro com dados válidos (RegisterViewModel)

Condição associada: CT09, CT11, CT14, CT17, CT20

Objetivo: Verificar se o sistema aceita o cadastro de um novo usuário quando todos os campos (nome, e-mail, senha, confirmação) são preenchidos corretamente.

Pré-condição: O RegisterViewModel foi instanciado; os controladores de texto estão limpos.

Dados de entrada:

- **nome:** "teste"
- **e-mail:** "teste@email.com"
- **senha:** "123456"
- **confirmação:** "123456"

TC-02 – Login com credenciais válidas (AuthServices – integração)

Condição associada: CT32

Objetivo: Confirmar que um usuário previamente cadastrado consegue realizar login com e-mail e senha corretos, recebendo um objeto de usuário autenticado.

Pré-condição: Uma conta foi criada com sucesso no Firebase através do

método createAccount. O utilizador fez logout em seguida.

Dados de entrada:

- **e-mail:** teste@gmail.com
- **password:** "123456"

Ação: Chamar auth.signIn(email: email, password: password).

Resultado esperado: O método retorna um UserCredential com a propriedade user não nula.

Tipo de teste: Integração.

TC-03 – Criação de LogPHR a partir de JSON (ContagemCortesService)

Condição associada: CT23

Objetivo: Verificar se o factory LogPHR.fromFirestore constrói corretamente um objeto a partir de um mapa JSON, interpretando data e hora.

Pré-condição: Nenhuma.

Dados de entrada:

```
{  
  "idSensor": "S1",  
  "leitura": 150,  
  "lote": 10,  
  "dataleitura": "2026-05-12",  
  "horaleitura": "14:30:00"  
}
```

Ação: Executar LogPHR.fromFirestore(json).

Resultado esperado:

- log.idSensor == "S1"
- log.leitura == 150
- log.lote == 10
- log.dataHora.year == 2026
- log.dataHora.hour == 14

Tipo de teste: Unidade

-

Condição associada: CT33

Objetivo: Assegurar que a tentativa de login com senha incorreta lança uma exceção.

Pré-condição: Conta criada com e-mail e senha "123456". Logout efetuado.

Dados de entrada:

- e-mail: mesmo e-mail da conta criada
- password: "999999"

Ação: Chamar `auth.signIn(email: email, password: "999999")` dentro de um bloco que capture exceções.

Resultado esperado: A chamada deve lançar uma exceção (espera-se `throwsException`).

Tipo de teste: Integração.

8.2 Tabela de Rastreabilidade de Casos de Teste

Prefixo e Módulo para rastreabilidade

RVM - RegisterViewmodel

LVM - LoginViewmodel

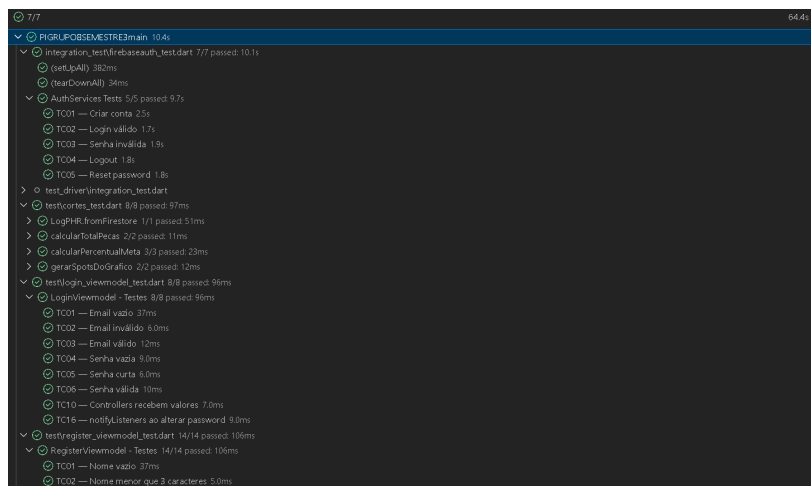
CCS - ContagemCortesService

AUTH - AuthService

Caso de Teste	Condição de Teste	Descrição resumida	Tipo
RVM-TC01	CT09	Nome vazio -> "nome é obrigatório"	Unidade
RVM-TC02	CT10	Nome curto (2 caracteres) -> mensagem de mínimo	Unidade
RVM-TC03	CT11	Nome válido -> retorna null	Unidade
RVM-TC04	CT12	E-mail vazio -> "Email é obrigatório"	Unidade
RVM-TC05	CT13	E-mail inválido -> "Digite um email válido"	Unidade
RVM-TC06	CT14	E-mail válido -> retorna null	Unidade
RVM-TC07	CT15	Senha vazia -> "Senha é obrigatório"	Unidade
RVM-TC08	CT16	Senha curta (3 caracteres) -> mensagem de mínimo	Unidade
RVM-TC09	CT17	Senha válida -> retorna null	Unidade
RVM-TC10	CT18	Confirmação vazia -> mensagem obrigatória	Unidade
RVM-TC11	CT19	Confirmação diferente da senha -> "As senhas não coincidem"	Unidade
RVM-TC12	CT20	Confirmação idêntica -> retorna null	Unidade
RVM-TC13	CT21	Alternar visibilidade da senha (<code>obscurePassword</code>)	Unidade
RVM-TC14	CT22	Alternar visibilidade da confirmação (<code>obscureConfirm</code>)	Unidade
LVM-TC01	CT01	E-mail vazio -> "Email é obrigatório"	Unidade
LVM-TC02	CT02	E-mail inválido -> "Digite um email válido"	Unidade
LVM-TC03	CT03	E-mail válido -> retorna null	Unidade

LVM-TC04	CT04	Senha vazia -> "Senha é obrigatório"	Unidade
LVM-TC05	CT05	Senha curta -> "A senha deve ter pelo menos 6 caracteres"	Unidade
LVM-TC06	CT06	Senha válida -> retorna null	Unidade
LVM-TC10	CT07	Controladores de texto recebem e armazenam valores	Unidade
LVM-TC16	CT08	togglePasswordVisibility notifica listeners	Unidade
CCS-TC01	CT23	LogPHR.fromFirestore cria objeto corretamente (data/hora)	Unidade
CCS-TC02	CT24	calcularTotalPecas soma leituras corretamente	Unidade
CCS-TC03	CT25	calcularTotalPecas retorna 0 com lista vazia	Unidade
CCS-TC04	CT26	calcularPercentualMeta retorna 0.5 para 1000 peças	Unidade
CCS-TC05	CT27	calcularPercentualMeta não ultrapassa 1.0 (5000 peças)	Unidade
CCS-TC06	CT28	calcularPercentualMeta retorna 0.0 quando total = 0	Unidade
CCS-TC07	CT29	gerarSpotsDoGrafico agrupa leituras por hora	Unidade
CCS-TC08	CT30	gerarSpotsDoGrafico retorna lista vazia sem logs	Unidade
AUTH-TC01	CT31	Criar conta no Firebase -> usuário não nulo	Integração
AUTH-TC02	CT32	Login com credenciais válidas -> usuário não nulo	Integração
AUTH-TC03	CT33	Login com senha inválida -> lança exceção	Integração
AUTH-TC04	CT34	Logout -> currentUser é nulo	Integração
AUTH-TC05	CT35	Reset de senha para e-mail existente -> execução sem exceção	Integração

9. Execução



test/register_viewmodel_test.dart 14/14 passed: 106ms

RegisterViewModel - Testes 14/14 passed: 106ms

TC01 — Nome vazio 37ms

TC02 — Nome menor que 3 caracteres 5.0ms

TC03 — Nome válido 6.0ms

TC04 — Email vazio 6.0ms

TC05 — Email inválido 6.0ms

TC06 — Email válido 5.0ms

TC07 — Senha vazia 5.0ms

TC08 — Senha curta 4.0ms

TC09 — Senha válida 4.0ms

TC10 — Confirmação vazia 8.0ms

TC11 — Senhas diferentes 4.0ms

TC12 — Confirmação válida 6.0ms

TC13 — Toggle password visibility 5.0ms

TC14 — Toggle confirm visibility 5.0ms